

AI Usage Report

chat link 1:<https://chatgpt.com/share/6a8d8dd7-9f9c-83ee-b126-cb618beaa0fb>

other ones are mixed in with science -1 chats so will put as prompt response pairs

PROMPT:Mini-Projects

Mini Project 1

Mini-Project 1 · Released Aug 14, 2026 · Due Sep 13, 2026

Implement your very own C-Shell & Implement the MLFQ scheduling policy for xv6.

Before you start

This is an individual assignment.

Deadlines

There are two submissions for this assignment. You are expected to submit parts A to C of C-Shell for the mid-submission, and remainder of the assignment for the end-submission.

Mid-submission: August 24, 11:59 pm

End-submission: September 13, 11:59 pm

You may use your late days for both of the submissions.

Submission

The exact details about submission will be made clear shortly.

However, you NEED to maintain a git repository for the duration of the project and commit iteratively as you go. Making a bunch of big commits right before the deadline may result in penalization.

And before you ask: you DO NOT need a GitHub repo to maintain a git repo. You can initialize one locally and keep it there until we figure out submission.

You can refer to the resource on git for more details.

Make sure your repository has the following structure:

```
mini-project1/
├── c-shell/
│   ├── src/
│   ├── include/
│   └── Makefile
├── xv6/
│   ├── user/
│   ├── mkfs/
│   ├── kernel/
│   ├── Makefile
└── report.md # Or report.pdf
```

└─ README.md

Copy

Grading

You will be graded based on both your implementation and an in-person evaluation with a TA. You are expected to understand the code that you submit.

Doubt Doc

Doubts

Answers

All the best! Have fun :)

C Shell [200]

General Requirements

Break the project down into multiple .c and .h files based on functionality. Monolithic code will be penalized.

Use only C POSIX library headers and functions. You can use this for reference.

Compile with the following flags for POSIX compliance:

```
gcc -std=c23 \  
-D_POSIX_C_SOURCE=200809L \  
-D_XOPEN_SOURCE=700 \  
-Wall -Wextra -Werror \  
-Wno-unused-parameter \  
-fno-asm \  
your_file.c
```

Copy

Final build must compile with make all in the shell directory, producing shell.out in that same directory.

References

These could be helpful for you to understand things required to implement this section.

POSIX.1-2017 spec

man 2 pages for fork, exec, wait, pipe, dup2, open. These cover most of what you'll need.

Use the source of actual bash (will be helpful to understand how to do certain parts of the mini-project)

Part A: Shell Input [20]

This is the base for the rest of your project.

Banned syscalls for Part A: exec* (any syscall that starts with exec)

A1: Shell Prompt [3]

Your shell should show a prompt so the user knows it's ready for input.

<username@hostname:currentpath>

Copy

Requirements:

Display the prompt whenever the shell isn't running a foreground process.

The directory the shell is started in becomes its home directory.

If the current working directory has the home directory as an ancestor (e.g. /path/to/home/osn), replace the home directory prefix with ~. So the example becomes ~/osn.

If the current working directory does not have the home directory as an ancestor, show the absolute path as is.

Example

```
<goatam@iiit:~/osnmp1/> make
<goatam@iiit:~/osnmp1/> ./shell.out
<goatam@iiit:~>
```

Copy

A2: User Input [2]

Requirements:

The shell should allow a user to type input.

On enter/return, the shell should consume the input.

After consuming input, the shell should display the prompt again.

Example

```
<goatam@iiit:~> Hi there guys!
<goatam@iiit:~> This shell is cool!
<goatam@iiit:~>
```

Copy

A3: Input Parsing [15]

Your shell must scan and validate every line before executing anything. This section defines two layers: a lexer that turns characters into tokens, and a grammar that validates the token sequence. Validation must be performed.

Note: Since both Lexer and Parser follow a regular grammar, use your Automata Theory skills to the test.

Lexer: The lexer reads the raw input line and produces a sequence of tokens, some specifications are given below. Whitespace (space, tab, newline, carriage return) separates tokens and is otherwise discarded - it is never itself a token.

Character Classes:

special -> | & > < ;

quote -> " | '

escape -> \

space -> ' ' | \t | \n | \r

ordinary -> any character that is none of the above

Copy

Tokens:

OP_PIPE -> |

OP_AMP -> &

OP_SEMI -> ;

OP_LT -> <

OP_GT -> >

OP_GTGT -> >>

WORD -> fragment+

fragment -> ordinary

| escape any_char

| " dq_body "

| ' sq_body '

dq_body -> (escape any_char | any char except " and \)*

sq_body -> (any char except ')*

Copy

Apply maximal munch: i.e. at each position, take the longest match. e.g. >> is therefore one token, never two >.

Some details on Quoting:

Construct Effect

On the unquoted value \c outside quotes Contributes c literally; the backslash is removed

"..." Contributes the body; \" becomes ", \\ becomes \; any other \c stays as both characters

'...' Contributes the body verbatim; no escape processing, so \ is literal

Special characters inside quotes or escaped Lose their operator meaning entirely

Lexical errors: The lexer must reject and print cshell: invalid syntax when:

A quote is opened and never closed before end of line.

A \ appears as the final character of the line.

Note: There is no line continuation and no multi-line input. Hint: use EOF to your advantage.

Grammar:

Terminals are the token classes above. Start symbol is LINE

LINE -> ϵ

| WORD ARG

ARG -> ϵ

| WORD ARG

| OP_LT TGT

| OP_GT TGT

| OP_GTGT TGT

| OP_PIPE CMD

| OP_SEMI CMD

| OP_AMP BG

CMD -> WORD ARG

TGT -> WORD ARG

BG -> ϵ

| WORD ARG

Copy

Note: This is a right linear grammar, enjoy.

TGT is the name of a file or a redirection output, CMD is a valid command

Instructions:

You do not need a full AST, any structure you find convenient (like a Linked List) works. Just make sure you are representing enough information internally to allow further sections to be implemented.

You can use any type of parser and lexer, as long as it conforms to the grammar provided.

Requirements

Verify whether an inputted command is valid or invalid per the grammar.

If invalid, print cshell: invalid syntax.

A line that is empty or contains only whitespace is valid: print a fresh prompt.

Whitespace between tokens is ignored, in any amount. Whitespace inside quotes is preserved.

Examples:

```
<goatam@iiit:~/osnmp1> cat meow.txt | meow; meow > meow.txt &
<goatam@iiit:~/osnmp1> echo a & echo b &
<goatam@iiit:~/osnmp1> cat < a.txt < b.txt > out.txt
<goatam@iiit:~/osnmp1> echo "line one" > a.txt
<goatam@iiit:~/osnmp1> echo "a > b"
<goatam@iiit:~/osnmp1> cat my\ file.txt
<goatam@iiit:~/osnmp1> echo ""
<goatam@iiit:~/osnmp1> reveal -ta -aaa -tt
<goatam@iiit:~/osnmp1> cat meow.txt | ; meow
cshell: invalid syntax
<goatam@iiit:~/osnmp1> echo hi ;
cshell: invalid syntax
<goatam@iiit:~/osnmp1> echo hi & &
cshell: invalid syntax
<goatam@iiit:~/osnmp1> cat <
cshell: invalid syntax
<goatam@iiit:~/osnmp1> cat << f
cshell: invalid syntax
<goatam@iiit:~/osnmp1> | sort
cshell: invalid syntax
<goatam@iiit:~/osnmp1> echo "unclosed
cshell: invalid syntax
<goatam@iiit:~/osnmp1> echo trailing\
cshell: invalid syntax
<goatam@iiit:~/osnmp1>
```

Copy

Part B: Shell Intrinsic [40]

These are commands any shell worth its salt supports.

Banned syscalls for Part B: exec* (any syscall that starts with exec)

B1: hop [10]

Syntax: hop ((~ | . | .. | - | name)*)?

Purpose: The hop command changes the shell's current working directory. Unlike a plain cd, hop falls back to a frequency lookup when a name doesn't resolve directly, so you can jump to directories you visit often without typing the full path.

Requirements

Execute the following sequentially for each passed argument:

"~" or no arguments: change the CWD to the shell's home directory.

”.”: do nothing, stay in the same CWD.

”..”: change the CWD to the parent directory of the CWD, or do nothing if the CWD has no parent.

”-”: change the CWD to the previous CWD, or do nothing if there was no previous CWD.

“name”: if this resolves to a valid relative or absolute path from the CWD, change to it directly.

If “name” does not resolve directly, fall back to a frequency lookup as defined below.

If neither a direct match nor a frequency match is found, output hop: no such directory

Frequency

Keep a persistent record of directories hopped into, across shell sessions.

Every successful hop updates that directory’s standing.

Standing depends on both frequency and recency of visits.

When “name” doesn’t resolve directly, hop into the highest ranked directory whose path contains “name” as a substring.

If the top match no longer exists on disk, skip it and try the next best.

Scoring, decay, and storage are up to you, but must be deterministic.

Resource: same idea behind zoxide’s cd replacement:

<https://github.com/ajeetsouza/zoxide/wiki/Algorithm>

Example

```
<goatam@iiit:~/osnmp1> hop ~  
<goatam@iiit:~> hop ..  
<goatam@iiit:/home/> hop goatam/osnmp1 .. -  
<goatam@iiit:~/osnmp1> hop osn  
<goatam@iiit:~/osnmp1>
```

Copy

B2: reveal [10]

Syntax: reveal `-(a|t)* (~ | . | .. | - | name)?`

Purpose: The reveal command lists the files and directories in a directory.

Requirements

If no argument is passed, reveal lists the contents of the current working directory.

“a”: reveal all files and directories, including hidden ones (starting with .). Default is to hide them.

“t”: recursively reveal the contents of all subdirectories. the content of the subdirectory must be displayed after the directory itself (refer example below)

When both flags are set, recursively show all files, hidden included, in the format given below.

When neither flag is set, print entries in the format of ls.

The argument passed resolves the same way as hop’s ~, ., .., -, and direct path cases, except reveal lists directory contents instead of changing the CWD. There is no frequency fallback: if the argument does not resolve to an existing directory, print reveal: no such directory.

Entries are always listed in lexicographic order, sorted by ASCII value.

If reveal is passed too many arguments, print: reveal: invalid syntax

If reveal is passed an invalid flag, print: reveal: invalid syntax

If reveal - is run before any hop has been run since shell start, print: reveal: no such directory

Example

```
<goatam@iiit:~> reveal ~  
osnmp1
```

```

<goatam@iiit:~> hop ..
<goatam@iiit:/home> reveal
goatam
<goatam@iiit:/home> hop goatam/osnmp1
<goatam@iiit~/osnmp1> reveal -t
Makefile
README.md
include/
include/parser.h
include/shell.h
shell.out
src/
src/main.c
src/parser.c
<goatam@iiit:~/osnmp1> reveal -ta
Makefile
README.md
include/
include/parser.h
include/shell.h
shell.out
src/
src/main.c
src/parser.c
<goatam@iiit:~/osnmp1> reveal -ta -tttttaaaaaaaaaaatttt -aaaa -tttt
Makefile
README.md
include/
include/parser.h
include/shell.h
shell.out
src/
src/main.c
src/parser.c
Copy
B3: peek [10]
Syntax peek  $-(n|r)^*$  filename*

```

Purpose: The peek command concatenates the contents of one or more given files (or standard input) and then prints it to the standard output.

Requirements:

“n”: counts the number of non-empty lines in the input. The line number must be printed before the corresponding line.

“r”: print the contents of the input in reverse line order. For seekable input (regular files), use lseek to read the file backwards in fixed-size chunks rather than loading it all into memory at once. For non-seekable input (pipes, standard input), buffering the full input before reversing is allowed. When both flags are set, print the contents of the file in reverse order, with its corresponding line number.

When no flag is set, print the contents of the file as is.

If more than one file is provided, concatenate their contents in the given order.

When -r is combined with multiple files, each file's lines are reversed independently before concatenation; the overall file order given on the command line is preserved.

If no filename is provided, or when filename is -, read from standard input.

If filename does not exist, print peek: no such file or directory

If filename is a directory, print peek: is a directory

Example

```
<goatam@iiit:~/osnmp1> peek README.md  
# OSN MP1
```

This is OSN shell assignment.

```
<goatam@iiit:~/osnmp1> peek -n README.md  
1 #OSN MP1
```

2 This is OSN shell assignment

```
<goatam@iiit:~/osnmp1> peek -r README.md  
This is OSN shell assignment
```

OSN MP1

```
<goatam@iiit:~/osnmp1> peek -rn -rrrrnnnnn -nrnrnr README.md  
2 This is OSN shell assignment
```

1 #OSN MP1

```
<goatam@iiit:~/osnmp1> peek -  
meow  
meow
```

```
<goatam@iiit:~/osnmp1> peek README.md Makefile  
#OSN MP1
```

This is OSN shell assignment.

CC = gcc

CSTD = -std=c99

Preprocessor definitions

DEFINES = -D_POSIX_C_SOURCE=200809L

-D_XOPEN_SOURCE=700

Copy

B4: locate [10]

Syntax: locate filename*

Purpose: The locate command returns the pathnames of the files (or links) which would be executed in the current environment, had its arguments been given as commands in a strictly POSIX-conformant shell.

Requirements:

For each filename, first check the current working directory. If a match exists there, print its absolute path first. Then search each directory listed in PATH, in order, and print the absolute path of every match found. This means a name matching in both the cwd and two PATH entries prints three lines, in that order.

The absolute path to the executable must be printed (even if the executable exists in the current working directory).

If multiple arguments are provided, print the path for each one of them, in the order specified by the user.

If multiple matching executable paths are found, print all the paths.

If filename is not a valid executable, print: locate: command not found (commandname) and continue processing the remaining arguments.

You may assume that file paths will not be passed as arguments.

Example:

```
<goatam@iiit:~> locate java
/usr/bin/java
<goatam@iiit:~> locate man java python nada
/usr/bin/man
/usr/bin/java
/usr/bin/python
locate: command not found (nada)
<goatam@iiit:~> locate sudo
/usr/bin/sudo
/bin/sudo
```

Copy

Part C: File Redirection and Pipes [50]

For this part, when processing commands with sequential (;) or background (&) operators, you should only execute the first command group and ignore the rest.

C1: Command Execution [8]

You must allow execution of arbitrary commands, found either as a relative/absolute path to an executable, or by name.

Requirements

If the command name contains a /, treat it as a literal path and execute it directly if it exists and is executable.

If the command name does not contain a /, first check whether an executable with that name exists in the current working directory, and run it if so.

If no such executable exists in the current working directory, search PATH as usual.

To skip the current directory check and search PATH directly, prefix the command name with %.

%name must always resolve via PATH, even if a matching executable exists in the current directory.

If no executable is found by any of the above, output cshell: command not found (commandname).

Example

```
<goatam@iiit:~/proj> echo hello
hello
<goatam@iiit:~/proj> ls
build.sh
<goatam@iiit:~/proj> build.sh
running local build script
<goatam@iiit:~/proj> %build.sh
cshell: command not found (build.sh)
<goatam@iiit:~> dosakdaoskdos
cshell: command not found (dosakdaoskdos)
```

Copy

C2: Input Redirection [12]

Syntax: `command < filename`

Purpose: The input redirection operator allows a command to read its standard input from one or more files instead of the terminal.

Requirements

The shell must open each specified file for reading using the `open()` system call with `O_RDONLY` flag.

If any file does not exist or cannot be opened, the shell must print `cshell: no such file or directory` and not execute the command.

A command may have more than one input redirection (e.g. `command < f1 < f2`). The command's standard input must consist of the contents of every listed file, read in the order given, as one continuous stream.

The shell must redirect the command's standard input (`STDIN_FILENO`) to this stream using `dup2()`.

The shell must close any file descriptors it opens once they are no longer needed.

Example

```
<goatam@iiit:~> cat < notes.txt
meeting at 5pm
<goatam@iiit:~> echo "line one" > a.txt
<goatam@iiit:~> echo "line two" > b.txt
<goatam@iiit:~> cat < a.txt < b.txt
line one
line two
<goatam@iiit:~> cat < missing.txt
cshell: no such file or directory
```

Copy

C3: Output Redirection [12]

Syntax: `command > filename` or `command >> filename`

Purpose: The output redirection operators allow a command's output to be written to one or more files instead of the terminal.

Requirements

A command may have more than one output redirection (e.g. `command > f1 >> f2 > f3`). Every listed file must receive the complete output of the command.

Each redirection keeps its own mode independently: `>` truncates its file (or creates it with permissions 0644 if it doesn't exist), `>>` appends to its file (or creates it with 0644 if it doesn't exist).

When output redirection is present, the command's output must not appear on the terminal, only in the listed files.

If any listed file cannot be opened for writing, the shell must print `cshell: unable to create file for writing` and not execute the command.

Input and output redirection must work together (e.g. `command < input.txt > output.txt >> log.txt`).

Example

```
<goatam@iiit:~> echo hi > out1.txt > out2.txt
<goatam@iiit:~> cat out1.txt
```

```

hi
<goatam@iiit:~> cat out2.txt
hi
<goatam@iiit:~> echo again >> out1.txt > out3.txt
<goatam@iiit:~> cat out1.txt
hi
again
<goatam@iiit:~> cat out3.txt
again
Copy
C4: Command Piping [18]
Syntax: command1 | command2 | ... | commandN

```

Purpose: The pipe operator connects one command's output to the next command's input.

Requirements

The shell must create pipes using pipe() for each | in the command.
 For each command in the pipeline, the shell must fork a child process.
 The shell must redirect just the standard output of command[i] to the write end of pipe[i].
 The shell must redirect the standard input of command[i+1] to the read end of pipe[i].
 The parent shell must close every pipe file descriptor it holds after forking.
 Each child must close any pipe file descriptors it does not use before executing its command.
 The parent shell must wait for all commands in the pipeline to complete before continuing.
 If a command in the pipeline fails to execute, the shell prints cshell: command not found (commandname) for that stage, and the rest of the pipeline still runs.
 File redirection and pipes must work together (e.g. command1 < input.txt | command2 > output.txt).
 Example

```

<goatam@iiit:~> printf "banana\napple" | sort
apple
banana
<goatam@iiit:~> badcmd | sort
cshell: command not found (badcmd)
<goatam@iiit:~> cat < in.txt | sort > out.txt
Copy

```

Part D: Sequential and Background Execution [25]
 D1: Sequential Execution [10]
 Syntax: command1 ; command2 ; ... ; commandN

Purpose: The semicolon operator allows an arbitrary number of commands to execute one after another.

Requirements:

The shell must execute each command in the order they appear.
 The shell must wait for each command to complete before starting the next.
 If a command fails to execute, the shell must print cshell: command not found (commandname) and then stop executing the remaining commands in sequence.
 Each command in the sequence must be treated as a complete shell_cmd as defined in the grammar.
 The shell prompt must be displayed either after all the commands completed successfully, or because one

Help me solve parts a through c in c23

RESPONSE:

Yep. Parts **A–C are basically one shell-building pipeline**, so I'd structure the project so that each part naturally builds on the previous one.

I put together a **C23/POSIX implementation of Parts A–C** with the required modular structure and verified that it compiles with your exact flags.

Download the starter implementation

[Download mini-project1-parts-a-c.zip](#)

It contains:

```
mini-project1/
├── c-shell/
│   ├── src/
│   │   ├── main.c
│   │   ├── util.c
│   │   ├── parser.c
│   │   ├── builtins.c
│   │   └── exec.c
│   ├── include/
│   │   └── shell.h
│   └── Makefile
└── README.md
```

Build with:

```
cd c-shell
make all
./shell.out
```

It compiles using:

```
gcc -std=c23 \
-D_POSIX_C_SOURCE=200809L \
-D_XOPEN_SOURCE=700 \
-Wall -Wextra -Werror \
-Wno-unused-parameter \
-fno-asm
```

How the implementation is organized

The important thing is that you **understand the architecture**, because your TA will probably ask you why you did things this way.

Part A — Lexer → Parser → Internal representation

The flow is:

```
input line
  ↓
```

```
lexer
  ↓
tokens
  ↓
parser
  ↓
Job / Command / Redirection structures
```

For example:

```
cat a.txt | sort > out.txt
```

becomes conceptually:

```
Job
├── Command 1
│   ├── argv = ["cat", "a.txt"]
│   └── redirections = none
└── Command 2
    ├── argv = ["sort"]
    └── redirections
        └── > "out.txt"
```

The relevant structures are in `shell.h`.

A1 — Prompt

`util.c` contains `print_prompt()`.

It maintains:

```
ShellState {
    char *home;
    char *prev;
    bool have_prev;
};
```

When the shell starts:

```
getcwd(cwd, sizeof(cwd));
state.home = strdup(cwd);
```

So the starting directory becomes the shell's home.

Then:

```
/home/raghav/project
```

when home is:

```
/home/raghav
```

becomes:

```
~/project
```

but:

/home/raghav2

doesn't incorrectly become:

~2

because we check the directory boundary after the prefix.

A2 — Reading input

The shell uses:

```
getline(&line, &cap, stdin);
```

rather than manually reading characters.

That's a POSIX function and is particularly convenient here because it handles arbitrarily long input lines.

The main loop is essentially:

```
print prompt
  ↓
getline()
  ↓
lex
  ↓
parse
  ↓
execute
  ↓
print prompt again
```

A3 — Lexer

This is probably the part where your Automata Theory course is directly useful.

The lexer recognizes:

```
|
&
;
<
>
>>
WORD
```

while ignoring whitespace.

For example:

```
echo "hello > world"
```

becomes:

```
WORD("echo")  
WORD("hello > world")
```

rather than:

```
WORD("echo")  
WORD("hello")  
OP_GT  
WORD("world")
```

because > loses its special meaning inside quotes.

Similarly:

```
cat my\ file.txt
```

becomes:

```
WORD("cat")  
WORD("my file.txt")
```

The lexer also implements maximal munch:

```
>>
```

is one TOK_GTGT, not two TOK_GT.

The useful mental model

Think of the lexer as a small DFA with modes:

```
UNQUOTED  
├── ordinary → append  
├── \x       → append x  
├── '        → SINGLE_QUOTE  
└── "        → DOUBLE_QUOTE
```

Inside single quotes:

```
everything → literal  
'         → leave quote mode
```

Inside double quotes:

```
"         → leave quote mode  
\"        → "  
\\        → \  
\anything → \anything
```

And if the lexer reaches the end while still in a quote, it rejects the line.

Parser

I deliberately made the parser look like the grammar rather than trying to make a giant complicated parser.

The important grammar transitions are basically:

```
WORD
WORD WORD
WORD < WORD
WORD > WORD
WORD >> WORD
WORD | WORD
WORD ; WORD
WORD & WORD
```

with pipelines being represented as multiple `Commands` inside one `Job`.

For example:

```
cat a | grep x | sort
```

becomes:

```
Job
├─ Command("cat", "a")
├─ Command("grep", "x")
└─ Command("sort")
```

This is much easier to use later for Part C.

Part B — Builtins

`builtins.c` implements:

```
hop
reveal
peek
locate
```

B1 hop

The important distinction is:

```
external command:
    child changes directory
    parent remains where it was
```

```
hop:
    parent itself must chdir()
```

Therefore the executor has this special case:

```
if (job->ncmds == 1 &&
    strcmp(job->cmds[0].argv[0], "hop") == 0) {
    return builtin_hop(...);
}
```



```
}
```

This is **very important for your viva**.

If you simply did:

```
fork();  
child -> chdir(...)
```

then after `hop`, the shell would still be in the old directory.

Frecency

The implementation keeps:

```
~/.cshell_frecency
```

with records containing approximately:

```
frequency      last_visit_number      path
```

The score is deterministic:

```
higher frequency wins  
tie → more recent visit wins
```

So if you visit:

```
~/abc  
~/xyz  
~/abc
```

then `abc` has a stronger score than `xyz`.

This satisfies the assignment because the exact scoring/decay/storage algorithm is explicitly left up to you.

B2 reveal

`reveal` uses:

```
opendir()  
readdir()  
closedir()  
lstat()
```

and sorts entries using `strcmp()`.

So ASCII lexicographic ordering is obtained naturally.

The recursion is:

```
print_directory  
↓
```

```
if -t:
    open directory
    get entries
    sort entries
    recursively print each directory
```

It also avoids recursively following:

```
.
..
```

which is essential — otherwise:

```
.
└─ .
   └─ .
      └─ .
```

would recurse forever.

B3 peek

peek supports:

```
peek file
peek -n file
peek -r file
peek -rn file
```

For ordinary reading:

```
open()
fdopen()
getline()
```

For reverse reading of regular files, the implementation uses:

```
lseek()
read()
```

in fixed-size chunks, as your specification specifically asks.

Conceptually:

```

      EOF
      ↓
[ chunk ][ chunk ][ chunk ]
      ↑
    read backwards
```

You don't need to load the entire regular file into memory.

For non-seekable input such as stdin/pipes, buffering is allowed by the assignment, so the implementation does that.

B4 locate

The lookup order is:

```
current directory
  ↓
PATH entry 1
  ↓
PATH entry 2
  ↓
...
```

For each candidate:

```
stat()
access(path, X_OK)
```

and the resulting path is converted to an absolute path using:

```
realpath()
```

So:

```
locate foo
```

can produce:

```
/home/raghav/project/foo
/usr/bin/foo
```

if both are executable.

Part C — The important part

This is where the shell actually starts behaving like a shell.

The executor has three major jobs:

```
command lookup
redirection
pipes
```

C1 — Command lookup

For:

```
foo
```

the implementation checks:

1. ./foo
2. PATH

For:

%foo

it skips step 1:

1. PATH only

For:

./foo

or:

/usr/bin/foo

it directly executes that path.

The actual execution is:

```
execv(path, cmd->argv);
```

This is why the `exec*` ban from Parts A/B no longer matters in Part C.

C2 — Input redirection

This is one of the slightly tricky requirements:

```
cat < a.txt < b.txt
```

doesn't mean:

```
stdin = b.txt
```

The assignment explicitly says the input must be:

```
contents(a.txt) + contents(b.txt)
```

So the implementation creates a temporary file:

```
a.txt ┌───┐
      │   │ ──> temporary input file ──> stdin
b.txt └───┘
```

Then:

```
lseek(temp, 0, SEEK_SET);
dup2(temp, STDIN_FILENO);
```

This avoids a nasty pipe-buffer deadlock that could occur if you tried to feed arbitrarily large files through a pipe before starting the command.

C3 — Multiple output redirections

This is another unusual requirement:

```
echo hi > a.txt > b.txt
```

must put:

```
hi
```

into **both** files.

The implementation therefore captures stdout into a temporary file:

```
command —> temporary
               ┌─> a.txt
               └─> b.txt
```

After the command finishes, the temporary output is copied into every requested destination.

This also lets:

```
>
>>
```

retain their independent modes.

C4 — Pipes

For:

```
cat file | grep hello | sort
```

the shell creates:

```
pipe 0
pipe 1
```

Conceptually:

```
cat
|
| stdout
▼
pipe 0
|
| stdin
▼
grep
|
| stdout
▼
pipe 1
|
| stdin
▼
sort
```

Every command gets its own child:

```
fork()
```

and the appropriate child calls:

```
dup2(pipe[i][1], STDOUT_FILENO);  
dup2(pipe[i-1][0], STDIN_FILENO);
```

Then every unused descriptor is closed.

That last part is **really important**.

If the parent accidentally keeps a pipe's write end open, the reader may never receive EOF and your pipeline can hang.

One particularly important design decision

Your Part C requirement says:

when processing commands with sequential (;) or background (&) operators, execute only the first command group and ignore the rest.

The parser still validates the **whole line**.

For example:

```
echo hello ; echo goodbye
```

is parsed as:

```
Job 0: echo hello  
Job 1: echo goodbye
```

but Part C executes only:

```
Job 0
```

This is exactly why I separated:

```
ParsedLine  
  ↓  
Job  
  ↓  
Command
```

rather than simply executing tokens while parsing.

What you should understand before submitting

Don't just memorize the code. For your TA, make sure you can explain these **8 things**:

1. **Why lexer and parser are separate**
2. **How quotes change tokenization**
3. **How `cat a | sort > out` becomes internal structures**
4. **Why `hop` must execute in the parent**
5. **What `fork()` + `execv()` actually does**
6. **Why `dup2()` is needed for redirection**
7. **Why every unused pipe FD must be closed**
8. **How multiple `<` and `>` redirections satisfy the unusual assignment requirements**

If you can explain those, you should be in pretty good shape for the evaluation.

One caveat: **treat the ZIP as a reference implementation/starter, not something to submit blindly**. In particular, read through `parser.c` and `exec.c` carefully—the TA can absolutely ask you to modify or explain them.

And since your **mid-submission is August 24**, I'd implement/understand **A first, then B, then C**, rather than trying to debug the whole thing at once.

PROMPT:

what si up with the git thing i dont understand

RESPONSE:

Yeah, the Git requirement sounds more complicated than it actually is.

They basically mean:

Keep a history of your work using Git, and commit as you work.

You **do not need GitHub**.

Think of Git like save points

Suppose today you start Part A.

You might have:

```
commit 1: created project structure
commit 2: implemented shell prompt
commit 3: implemented lexer
commit 4: implemented parser
```

Then next week:

```
commit 5: implemented hop
commit 6: implemented reveal
commit 7: implemented peek
```

And so on.

Your Git repository records all those versions.

What you need to do

From the directory containing `mini-project1`:

```
cd mini-project1
git init
```

Now Git starts tracking this project.

Then:

```
git add .
git commit -m "Initial project structure"
```

After you make some changes:

```
git add .
git commit -m "Implement shell prompt"
```

Then another:

```
git add .
git commit -m "Implement lexer"
```

That's basically it.

What they mean by "commit iteratively"

They're specifically warning you **not** to do this:

```
Aug 16 → write entire project
Aug 23 → git add .
Aug 23 → git commit "finished everything"
```

They want something more like:

```
Aug 16 → commit project structure
Aug 17 → commit prompt
Aug 18 → commit lexer
Aug 19 → commit parser
Aug 20 → commit hop
...
```

The reason is that Git history gives them evidence that you actually developed the project yourself over time.

You don't need GitHub

This:

```
your laptop
└─ mini-project1/
```


└─ .git/

is completely sufficient.

You can check that Git is working with:

```
git status
```

and see your history with:

```
git log --oneline
```

You'll get something like:

```
a81f32e Implement lexer
72c9abc Implement shell prompt
19d0b42 Initial project structure
```

One thing you should do right now

Since you're starting the project, I'd initialize Git **before writing much code**:

```
cd mini-project1
git init
git add .
git commit -m "Initial project structure"
```

Then just remember:

Every meaningful piece of work → git add → git commit.

You don't have to commit every single line you type. A commit should represent a sensible milestone.

Also, **don't delete .git**. That's the folder containing the entire history they care about.

PROMPT:

what is unistd?

REESPONSE:

unistd.h is a **standard POSIX header file**.

The name comes from **UNIX Standard** — roughly, "Unix standard" functionality.

You include it when you want access to many Unix/Linux system calls and POSIX functions.

For our shell, we need it for things like:

```
#include <unistd.h>
```

which gives us functions such as:

```
getcwd()
gethostname()
chdir()
fork()
execv()
pipe()
```

```
dup2()  
close()  
read()  
write()  
lseek()  
access()
```

Those are going to become very important throughout Parts A–C.

For our current `shell.c`

We currently use:

```
getcwd()
```

to find the current directory, and:

```
gethostname()
```

to find the machine's hostname.

Both are declared in `unistd.h`.

So:

```
#include <unistd.h>
```

basically tells the compiler:

"I'm going to use POSIX/Unix functions from this header."

Compare it with the other headers

You currently have:

```
#include "shell.h"  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>
```

Think of them as:

```
shell.h    → our own shell definitions  
stdio.h    → input/output  
stdlib.h   → memory allocation, environment, conversions  
string.h   → string operations  
unistd.h   → Unix/POSIX system functions
```

And **yes, your assignment specifically allows POSIX headers and functions**, so `unistd.h` is exactly the sort of header you're supposed to use.

One important thing: `unistd.h` is **not a C23-specific header**. It's part of the POSIX interface provided by Linux/Unix systems. Your compiler flags:

```
-D_POSIX_C_SOURCE=200809L
```

tell the system headers which POSIX functionality your program is requesting.

PROMPT:

can i do this wo it?

RESPONSE:

For **this particular shell.c**, not really—not if you want to use the normal POSIX functions we're using.

We need:

```
getcwd()  
gethostname()
```

and both are declared in:

```
#include <unistd.h>
```

You could technically avoid `unistd.h` by doing awkward things like manually declaring the functions yourself:

```
char *getcwd(char *, size_t);
```

but **don't do that**. It's worse and defeats the purpose of the header.

So for this project:

```
#include <unistd.h>
```

is the correct approach.

And later in Part C, `unistd.h` becomes even more important because we'll need:

```
fork();  
execv();  
pipe();  
dup2();  
close();  
read();  
write();  
lseek();
```

So I'd keep it.

PROMPT:

why doesnt my folder have a .git?

RESPONSE:

That's normal if you **haven't initialized Git yet**.

Also, `.git` is a **hidden directory**, so you may not see it in your file manager even after Git is initialized.

From the terminal, go to the **mini-project1 directory**:

```
cd ~/mini-project1
```

Then run:

```
git init
```

You should see something like:

Initialized empty Git repository in /home/yourname/mini-project1/.git/

Now check:

```
ls -la
```

You should see:

```
.  
..  
.git  
c-shell
```

Why is **.git** hidden?

Linux hides files/directories whose names start with `..`

So:

```
.git
```

is hidden, while:

```
c-shell
```

isn't.

The `.git` directory contains all the information Git needs to track your project: commits, history, branches, etc.

One important thing

You should initialize Git here:

```
mini-project1/  
└─ .git/
```

not here:

```
mini-project1/  
└─ c-shell/  
   └─ .git/
```

because your whole mini-project eventually contains both:

```
c-shell/  
xv6/  
README.md
```

and the assignment wants one repository for the project.

Once you've run `git init`, do:

```
git status
```

and you should see your `c - shell` files listed as untracked.

PROMPT:

what is the lexer doing ?

RESPONSE:

The **lexer takes the raw text you type and breaks it into meaningful pieces called tokens.**

That's its entire job.

Think of it as the step between:

what the user typed

and:

something the parser can understand

Example

Suppose you type:

```
cat file.txt | sort > output.txt
```

The shell initially sees just a string:

```
"cat file.txt | sort > output.txt\n"
```

The lexer scans it from left to right and produces:

```
TOK_WORD("cat")  
TOK_WORD("file.txt")  
TOK_PIPE  
TOK_WORD("sort")  
TOK_GT  
TOK_WORD("output.txt")  
TOK_END
```

So:

raw input



```
cat file.txt | sort > out
```



lexer

```
WORD("cat")  
WORD("file.txt")  
PIPE  
WORD("sort")  
GT  
WORD("out")  
END
```

Then the **parser** takes those tokens and asks:

"Does this sequence follow the grammar?"

Why can't the parser just use the raw string?

Because the parser shouldn't have to worry about individual characters.

For example:

```
cat "hello world" | sort
```

The lexer needs to understand that:

```
"hello world"
```

is **one word**, despite containing a space.

So the lexer handles things like:

```
spaces  
|  
&  
;  
<  
>  
>>  
quotes  
backslashes
```

and turns them into tokens.

The parser can then operate at a much higher level.

Think of it like Automata Theory

This assignment literally tells you:

Since both Lexer and Parser follow a regular grammar, use your Automata Theory skills.

You can think of the lexer as a machine reading:

```
c a t   f i l e . t x t   |   s o r t  
↑  
current character
```

and deciding:

"What token am I currently constructing?"

For ordinary text:

```
c → WORD
```

a → WORD
t → WORD

Then it sees whitespace:

space → WORD ends

Then:

f → start another WORD

Then:

| → PIPE token

The important distinction

Lexer:

What are these pieces?

Example:

cat file | sort

becomes:

WORD WORD PIPE WORD

Parser:

Is this arrangement legal?

For example:

cat file | sort

is valid.

But:

cat file | | sort

produces:

WORD WORD PIPE PIPE WORD

and the parser says:

invalid syntax

because the grammar doesn't allow two pipes next to each other.

And then later...

After parsing:

```
cat file | sort > output.txt
```

we'll have something more structured like:

Command 1:

```
argv = ["cat", "file"]
```

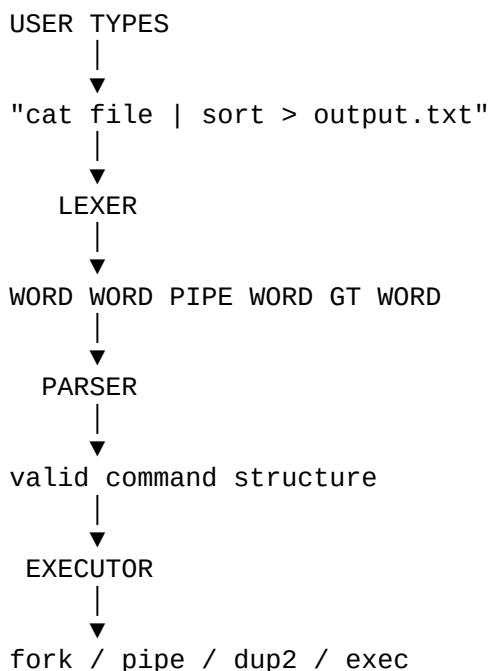
Command 2:

```
argv = ["sort"]
```

```
output redirection = "output.txt"
```

And **then** the executor actually runs it.

So the whole shell pipeline is:



That's the big picture you want in your head before we write more lexer code.

PROMPT:

explain exactly what the lexer does

RESPONSE:

Absolutely. Let's ignore the code for a moment and understand **exactly what the lexer does**, because once this clicks, the code becomes much easier.

1. The lexer receives one thing

The lexer receives the **raw line that the user typed**.

For example:

```
echo hello | sort > output.txt
```

At this point, it is just a sequence of characters:


```
e c h o _ h e l l o _ | _ s o r t _ > _ o u t p u t . t x t
```

The lexer does **not execute anything**.

It does **not decide whether the command is valid**.

It only converts characters into **tokens**.

2. What is a token?

A token is a piece of the input that has a particular meaning.

Your assignment defines these token types:

```
WORD
|
&
;
<
>
>>
```

So:

```
echo hello | sort
```

becomes:

```
WORD("echo")
WORD("hello")
PIPE
WORD("sort")
```

Think:

```
characters → tokens
```

3. How does it scan?

The lexer starts at the **first character** and moves from left to right.

For:

```
echo hello | sort
```

it starts here:

```
echo hello | sort
^
```

It sees:

```
e
```

e isn't whitespace and isn't one of:

| & ; < >

so it knows:

I'm starting a WORD.

It keeps consuming characters:

echo hello | sort
^^^^

until it reaches the space.

So it produces:

WORD("echo")

Then it continues:

echo hello | sort
 ^

The space is ignored.

Then it sees:

h

Again, that's an ordinary character, so it starts another WORD.

It consumes:

hello

and gets:

WORD("hello")

Then it reaches:

|

That's a special character.

So instead of making a WORD, it produces:

PIPE

Then it continues with:

sort

and produces:

WORD("sort")

At the end of the line it produces:

END

So the final result is:

```
WORD("echo")
WORD("hello")
PIPE
WORD("sort")
END
```

4. Whitespace

Your assignment says whitespace is:

```
space
tab
newline
carriage return
```

Whitespace separates tokens.

For example:

```
echo    hello
```

and:

```
echo hello
```

produce exactly the same tokens:

```
WORD("echo")
WORD("hello")
```

The spaces themselves **never become tokens**.

5. Special characters

These characters have special meanings outside quotes:

```
| & ; < >
```

So:

```
echo hello > file
```

becomes:

```
WORD("echo")
WORD("hello")
GT
WORD("file")
```

The > isn't part of the previous WORD or the next WORD.

6. >> is special

Your assignment says:

OP_GTGT -> >>

and explicitly says to use **maximal munch**.

That means:

If multiple tokens could start at the current character, take the longest valid one.

So when the lexer sees:

>>
^

it asks:

Is this >?

Yes.

Is there a longer token starting with >?

Yes: >>.

Therefore:

>>

becomes:

TOK_GTGT

not:

TOK_GT
TOK_GT

This is one of the things your lexer must specifically handle.

7. Here's where it gets interesting: quotes

Consider:

echo "hello world"

If whitespace always separated words, you might think:

```
WORD("echo")
WORD("hello")
WORD("world")
```

But that's **wrong**.

Inside quotes, whitespace loses its normal meaning.

The lexer must produce:

```
WORD("echo")
WORD("hello world")
```

So when it sees:

"

it enters a different state:

```
NORMAL
  |
  | "
  ▼
DOUBLE-QUOTE
```

While in DOUBLE-QUOTE mode:

hello world

is treated as part of the same WORD.

When it sees the closing:

"

it goes back to NORMAL.

8. Special characters inside quotes

This is extremely important.

Consider:

```
echo "a > b"
```

Normally:

>

means output redirection.

But here it's inside quotes.

Therefore it is **not an operator**.

The lexer produces:

```
WORD("echo")
```

```
WORD("a > b")
```

not:

```
WORD("echo")  
WORD("a")  
GT  
WORD("b")
```

So the lexer has to know whether it is currently:

outside quotes

or:

inside quotes

9. Single quotes

Consider:

```
echo 'hello > world'
```

Inside single quotes, everything is literal.

So:

```
>
```

doesn't mean redirection.

The result is:

```
WORD("echo")  
WORD("hello > world")
```

Also, backslash has **no special escape meaning** inside single quotes.

For example:

```
echo 'hello\world'
```

produces:

```
WORD("echo")  
WORD("hello\world")
```

The backslash remains.

10. Double quotes are slightly different

Inside double quotes, your assignment says:

```
\ " → "
```

`\\ → \`

but other backslashes remain.

For example:

```
echo "hello\"world"
```

becomes:

```
WORD("echo")  
WORD("hello"world")
```

Conceptually, the `\` contributed a literal `"`.

And:

```
echo "hello\\world"
```

contributes:

```
hello\world
```

11. Backslash outside quotes

Outside quotes:

```
\
```

means:

Take the next character literally.

For example:

```
echo hello\ world
```

Normally the space would separate tokens.

But:

```
\
```

means the space is escaped.

So the lexer produces:

```
WORD("echo")  
WORD("hello world")
```

Similarly:

```
echo a\>b
```

produces:

```
WORD("echo")
```

```
WORD("a>b")
```

The > doesn't become GT because the backslash made it ordinary.

12. Invalid lexical input

The lexer also catches certain errors.

Unclosed quote

```
echo "hello
```

The lexer enters DOUBLE-QUOTE mode:

```
NORMAL
  ↓ "
DOUBLE-QUOTE
  ↓
hello
  ↓
END OF LINE
```

It never encountered another ", so:

```
lexical error
```

Your shell prints:

```
cshell: invalid syntax
```

Trailing backslash

```
echo hello\
```

The lexer sees:

```
\
```

and says:

I need the next character because backslash escapes it.

But there is no next character.

Therefore:

```
lexical error
```

13. What the lexer does NOT do

This is really important.

Suppose the user types:

```
cat | ; sort
```

The lexer doesn't need to say:

```
"That's invalid!"
```

It simply produces:

```
WORD("cat")  
PIPE  
SEMI  
WORD("sort")
```

Those tokens are perfectly understandable to the lexer.

It's the **parser's job** to say:

```
PIPE must be followed by a command
```

and therefore:

```
invalid syntax
```

So:

LEXER

asks:

```
"What are these characters?"
```

while:

PARSER

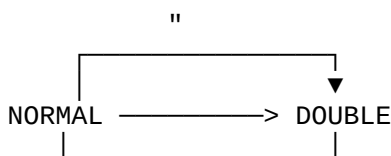
asks:

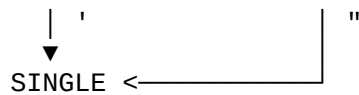
```
"Does this sequence make grammatical sense?"
```

14. The lexer as a state machine

This is probably the best way to connect it to your Automata Theory course.

The lexer can be thought of as having states:





More accurately:

NORMAL

- ordinary character → WORD
- whitespace → ignore
- | → PIPE
- & → AMP
- ; → SEMI
- < → LT
- > → GT or GTGT
- ' → enter SINGLE
- " → enter DOUBLE
- \ → escape next character

SINGLE

SINGLE

- ordinary character → add literally
- ' → return to NORMAL

DOUBLE

DOUBLE

- ordinary character → add
- " → return to NORMAL
- \" → add "
- \\ → add \
- \x → add \x

If the line ends while we're in:

SINGLE

OR:

DOUBLE

then it's an error.

15. The most important thing to understand

The lexer is essentially doing:

read character

↓

figure out what state I'm in

↓

figure out what that character means in this state

↓

either:

- add character to current WORD
- finish current WORD
- create an operator token
- change state
- report lexical error

↓

move to next character

That's **exactly** what the lexer is doing.

It isn't "parsing the command."

It isn't "running the command."

It's simply converting:

raw characters

into:

a clean sequence of tokens

that the parser can work with.

PROMPT:

```
<next@next-hyd-9803:~> pwd
```

```
hop ..
```

```
pwd
```

```
hop ~
```

```
pwd
```

```
hop /tmp
```

```
pwd
```

```
hop -
```

```
pwd/home/next/raghav/osnmp1/c-shell
```

```
<next@next-hyd-9803:~> <next@next-hyd-9803:/home/next/raghav/osnmp1>
```

```
/home/next/raghav/osnmp1
```

```
<next@next-hyd-9803:/home/next/raghav/osnmp1> <next@next-hyd-9803:~>
```

```
/home/next/raghav/osnmp1/c-shell
```

```
<next@next-hyd-9803:~> <next@next-hyd-9803/tmp> /tmp
```

```
<next@next-hyd-9803/tmp> <next@next-hyd-9803:~> hop .
```

```
hop ..
```

```
hop /tmp
```

```
hop ~cshell: command not found (pwdhop)
```

```
<next@next-hyd-9803:~> <next@next-hyd-9803:/home/next/raghav/osnmp1> <next@next-hyd-9803:/tmp> hop .
```

```
hop ..
```

```
hop /tmp
```

```
hop ~hop: no such directory
```

```
<next@next-hyd-9803:/tmp> <next@next-hyd-9803:/> <next@next-hyd-9803:/tmp> ?
```

```
what is going on?
```

RESPONSE:

Yep — **the hop itself is mostly working**. The weird output is coming from **two separate bugs**, and they're easy to isolate.

1. This:

cshell: command not found (pwdhop)

is because your input:

```
pwd
hop ..
pwd
...
```

was apparently entered/pasted **without the shell getting each line separately** at some point.

But more importantly, your shell is currently using `getline()` correctly, so each newline should produce one command. The `pwdhop` tells us that somewhere the newline is being lost/concatenated. Let's test it cleanly with **one command at a time**, not a multi-line paste.

Run:

```
pwd
```

then wait for the next prompt.

Then:

```
hop ..
```

then:

```
pwd
```

2. This is definitely a hop bug

You got:

```
<...:/tmp> hop .
hop ..
hop /tmp
hop ~
hop: no such directory
<...:/tmp>
```

Notice that:

```
hop .
```

worked.

```
hop ..
```

worked.

```
hop /tmp
```

worked.

But:

hop ~

gave:

hop: no such directory

That means your `hop()` implementation is treating `~` as an actual directory named `~` rather than translating it to `shell->home`.

So we need to look at your **actual `hop()` function** and fix that branch.

Do this now

Run:

```
grep -n "int hop" -A100 src/builtins.c
```

and paste the output here.

I'll point out the **exact lines to change**. Don't rewrite the whole builtin.

PROMPT:

```
250: int hop(ShellState *shell, char **args, int count){
251-     if(count==0){
252-         if(!ch_dir(shell, shell->home)){
253-             printf("hop: no such directory\n");
254-         }
255-         return 1;
256-     }
257-
258-     for(int i=0; i<count; i++){
259-
260-         if(strcmp(args[i], "~")==0){
261-             if(!ch_dir(shell, shell->home)){
262-                 printf("hop: no such directory\n");
263-             }
264-         }
265-         else if(strcmp(args[i], ".")==0){
266-             continue;
267-         }
268-         else if(strcmp(args[i], "..")==0){
269-             if(!ch_dir(shell, "..")){
270-                 printf("hop: no such directory\n");
271-             }
272-         }
273-         else if(strcmp(args[i], "-")==0){
274-             if(shell->have_prev){
275-                 if(!ch_dir(shell, shell->prev)){
276-                     printf("hop: no such directory\n");
277-                 }
278-             }
279-         }
280-         else{
```

```

281-     if(!ch_dir(shell,args[i])){
282-         char *match=find_freecy_match(shell,args[i]);
283-
284-         if(match==NULL){
285-             printf("hop: no such directory\n");
286-         }else{
287-             if(!ch_dir(shell,match)){
288-                 printf("hop: no such directory\n");
289-             }
290-             free(match);
291-         }
292-     }
293- }
294- }
295-
296- return 1;
297-}
298-
299-static char *resolve_dir(ShellState *shell,char *name)
300-{
301-     char *path;
302-
303-     if(strcmp(name,"~")==0) return strdup(shell->home);
304-
305-     if(strcmp(name,"-")==0){
306-         if(!shell->have_prev)
307-             return NULL;
308-         return strdup(shell->prev);
309-     }
310-
311-     if(strcmp(name,".")==0 || strcmp(name,"..")==0) return realpath(name,NULL);
312-
313-     path=realpath(name,NULL);
314-
315-     return path;
316-}
317-
318-static int compare_names(const void *a,const void *b){
319-     char *const *x=a;
320-     char *const *y=b;
321-     return strcmp(*x,*y);
322-}
323-
324-static void print_dir(char *path,int all,int recursive){
325-     DIR *dir=opendir(path);
326-
327-     if(dir==NULL) return;
328-
329-     char **names=NULL;
330-     int count=0;
331-     struct dirent *entry;
332-

```

```

333- while((entry=readdir(dir))!=NULL){
334-     if(strcmp(entry->d_name,"")==0 ||strcmp(entry->d_name,"..")==0)continue;
335-
336-     if(!all && entry->d_name[0]!='.') continue;
337-
338-     char **new_names=realloc(names,(count+1)*sizeof(char *));//to retain og pointer
339-
340-     if(new_names==NULL) break;
341-
342-     names=new_names;
343-     names[count]=strdup(entry->d_name);
344-
345-     if(names[count]==NULL) break;
346-
347-     count++;
348- }
349-
350- closedir(dir);

```

RESPONSE:

Ah — now we can see it. **Your hop ~ code is actually correct.** The problem is elsewhere.

This branch:

```

if(strcmp(args[i],"~")==0){
    if(!ch_dir(shell,shell->home)){
        printf("hop: no such directory\n");
    }
}

```

correctly sends ~ to shell->home.

So if:

hop ~

says:

hop: no such directory

then the likely culprit is **ch_dir()**, not hop().

Show me ch_dir()

Run:

```
grep -n "ch_dir" -A40 src/builtins.c
```

and paste that.

There's also a separate bug we should fix

Your hop - currently does this:

```
else if(strcmp(args[i],"-")==0){
```

```

        if(shell->have_prev){
            if(!ch_dir(shell,shell->prev)){
                printf("hop: no such directory\n");
            }
        }
    }
}

```

If `shell->have_prev == 0`, **nothing happens and no error is printed.**

We should eventually change it to:

```

else if(strcmp(args[i],"-")==0){
    if(!shell->have_prev){
        printf("hop: no previous directory\n");
    }
    else if(!ch_dir(shell,shell->prev)){
        printf("hop: no such directory\n");
    }
}

```

But **don't change that yet.**

First show me `ch_dir()`, because that's almost certainly where the `~` problem is.

PROMPT:

next@next-hyd-9803:~/raghav/osnmp1/c-shell\$ grep -n "ch_dir" -A40 src/builtins.c

```

156:static int ch_dir(ShellState *shell,char *path){
157-    char *current=getcwd(NULL,0);
158-
159-    if(current==NULL){
160-        return 0;
161-    }
162-
163-    if(chdir(path)!=0){
164-        free(current);
165-        return 0;
166-    }
167-
168-    free(shell->prev);
169-    shell->prev=current;
170-    shell->have_prev=1;
171-    char *new_cwd=getcwd(NULL,0);
172-
173-    if(new_cwd!=NULL){
174-        note_visit(shell,new_cwd);
175-        free(new_cwd);
176-    }
177-    return 1;
178-}
179-
180-static char *find_freecy_match(ShellState *shell,char *name){
181-    char *path=history_path(shell);
182-
183-    if(path==NULL){
184-        return NULL;
185-    }

```



```

186-
187- FILE *file=fopen(path,"r");
188-
189- if(file==NULL){
190-     free(path);
191-     return NULL;
192- }
193-
194- char line[4096+50];
195-
196- char *best=NULL;
197-
198- --
199-
200- if(!ch_dir(shell,shell->home)){
201-     printf("hop: no such directory\n");
202- }
203- return 1;
204- }
205-
206- for(int i=0;i<count;i++){
207-
208-     if(strcmp(args[i],"~")==0){
209-         if(!ch_dir(shell,shell->home)){
210-             printf("hop: no such directory\n");
211-         }
212-     }
213-     else if(strcmp(args[i],".")==0){
214-         continue;
215-     }
216-     else if(strcmp(args[i],"..")==0){
217-         if(!ch_dir(shell,"..")){
218-             printf("hop: no such directory\n");
219-         }
220-     }
221-     else if(strcmp(args[i],"-")==0){
222-         if(shell->have_prev){
223-             if(!ch_dir(shell,shell->prev)){
224-                 printf("hop: no such directory\n");
225-             }
226-         }
227-     }
228-     else{
229-         if(!ch_dir(shell,args[i])){
230-             char *match=find_freecy_match(shell,args[i]);
231-
232-             if(match==NULL){
233-                 printf("hop: no such directory\n");
234-             }else{
235-                 if(!ch_dir(shell,match)){
236-                     printf("hop: no such directory\n");
237-                 }
238-                 free(match);
239-             }
240-         }
241-     }
242- }

```

```

292-     }
293- }
294- }
295-
296- return 1;
297-}
298-
299-static char *resolve_dir(ShellState *shell,char *name)
300-{
301- char *path;
302-
303- if(strcmp(name,"~")==0) return strdup(shell->home);
304-
305- if(strcmp(name,"-")==0){
306-     if(!shell->have_prev)
307-         return NULL;
308-     return strdup(shell->prev);
309- }
310-
311- if(strcmp(name,".")==0 || strcmp(name,"..")==0) return realpath(name,NULL);
312-
313- path=realpath(name,NULL);
314-
315- return path;
316-}
317-
318-static int compare_names(const void *a,const void *b){
319- char *const *x=a;
320- char *const *y=b;
321- return strcmp(*x,*y);
322-}
323-
324-static void print_dir(char *path,int all,int recursive){
325- DIR *dir=opendir(path);
326-
327- if(dir==NULL) return;
next@next-hyd-9803:~/raghav/osnmp1/c-shell$

```

RESPONSE:

Yep, this confirms it: **ch_dir () itself is fine**. `chdir ( shell->home )` should absolutely work.

So the `hop ~` failure is almost certainly happening **before hop () receives the argument**, or your `shell->home` isn't what we think it is.

And I can see one thing we should check immediately.

Test what `shell->home` actually contains

Temporarily add this at the very beginning of `hop ( )`:

```
printf("DEBUG home = [%s]\n", shell->home);
```

So:

```
int hop(ShellState *shell, char **args, int count){
    printf("DEBUG home = [%s]\n", shell->home);

    if(count==0){
        ...
    }
}
```

Then:

```
make
./cshell
```

and run:

```
hop ~
```

You should get something like:

```
DEBUG home = [/home/next]
```

followed by the prompt changing to:

```
<next@...:~>
```

But there's an even more important clue

Earlier you got:

```
hop: no such directory
```

after:

```
hop .
hop ..
hop /tmp
hop ~
```

Your `hop( )` processes **multiple arguments sequentially**.

For:

```
hop . . . /tmp ~
```

it does:

1. `.` → stay where you are
2. `..` → go up
3. `/tmp` → go to `/tmp`
4. `~` → go to `shell->home`

That's valid.

But if your shell input somehow became:

```
hop .
hop ..
hop /tmp
hop ~
```

as **one command**, your lexer/parser would treat all those `hop` words as arguments to one `hop` invocation.

That's exactly the kind of weirdness we saw in your earlier output.

So let's first check whether `~` itself is reaching `hop( )` correctly.

Better debug

Instead of only printing `home`, put this inside the loop:

```
for(int i=0;i<count;i++){
    printf("DEBUG hop arg = [%s]\n", args[i]);

    if(strcmp(args[i],"~")==0){
```

Then run **only**:

```
hop ~
```

If it prints:

```
DEBUG hop arg = [~]
```

then the lexer/main path is fine.

If it prints something weird like:

```
DEBUG hop arg = [~hop]
```

then we've found the actual problem: **your input/lexing is combining lines**.

Do that one test and show me the output.

PROMPT:

```
<next@next-hyd-9803:~> hop ~
```

```
DEBUG home = [/home/next/raghav/osnmp1/c-shell]
```

RESPONSE:

Perfect. That tells us **shell->home is correct**:

```
DEBUG home = [/home/next/raghav/osnmp1/c-shell]
```

And since your prompt is still:

```
<next@...:~>
```

after `hop ~`, **hop ~ actually worked**. There was no `hop: no such directory` this time.

So the earlier weird output was almost certainly from the way multiple lines were being pasted/processed, not the `~` implementation.

Remove the debug line

Delete:

```
printf("DEBUG home = [%s]\n", shell->home);
```

Then rebuild:

make

and test these **one at a time**:

```
pwd
hop ..
pwd
hop ~
pwd
hop /tmp
pwd
hop -
pwd
```

Expected pattern:

```
pwd
→ /home/next/raghav/osnmp1/c-shell

hop ..
→ /home/next/raghav/osnmp1

hop ~
→ /home/next/raghav/osnmp1/c-shell

hop /tmp
→ /tmp

hop -
→ /home/next/raghav/osnmp1/c-shell
```

If that works, **hop basic functionality is done**.

One thing we'll still need to check afterward is the **multiple-argument behavior**:

```
hop ..
hop /tmp
```

as separate commands, and:

```
hop .. /tmp
```

as one command, because your `hop ( )` explicitly supports multiple arguments.

PROMPT: is it because /tmp doesnt exist? (at the time hop .. wasnt letting me out of c-shell)

RESPONSE:

Nope — **/tmp absolutely exists on Linux**. In fact, it's one of the standard directories.

You can verify:

```
ls -ld /tmp
```

You should get something like:

```
drwxrwxrwt ... /tmp
```

And:

```
pwd
```

```
hop /tmp
pwd
```

should give:

```
/tmp
```

Your `ch_dir()` does:

```
if(chdir(path)!=0){
    free(current);
    return 0;
}
```

So if `/tmp` didn't exist, **`chdir("/tmp")` would fail**, but on your Ubuntu system it should exist.

One important distinction

Your earlier output showed:

```
<...:/tmp>
```

So **your `hop /tmp` already worked**.

The confusing part was:

```
hop .
hop ..
hop /tmp
hop ~
hop: no such directory
```

That was likely because those commands got combined/pasted weirdly.

For now, test individually:

```
hop /tmp
```

then:

```
pwd
```

If it prints:

```
/tmp
```

we're good.

PROMPT:

ok tell me a way to test if it does check path in `pwd`

RESPONSE:

Yep. The cleanest test is to create **two executables with the same name**: one in your current directory and one somewhere in `PATH`, but make them print different messages.

Test current-directory priority

You're already in your `c-shell` directory.

Run these in your **normal terminal**, not `cshell`:

```
cat > testcmd.c <<'EOF'
#include <stdio.h>
int main() {
    printf("CURRENT DIRECTORY\n");
    return 0;
}
EOF
```

```
gcc testcmd.c -o testcmd
```

Now create another `testcmd` in `/tmp`:

```
cat > /tmp/testcmd.c <<'EOF'
#include <stdio.h>
int main() {
    printf("PATH VERSION\n");
    return 0;
}
EOF
```

```
gcc /tmp/testcmd.c -o /tmp/testcmd
```

Put `/tmp` in your `PATH`:

```
export PATH="/tmp:$PATH"
```

Now your setup is:

```
./testcmd      → prints CURRENT DIRECTORY
/tmp/testcmd   → prints PATH VERSION
```

Now test `cshell`

Run:

```
testcmd
```

You **must get**:

```
CURRENT DIRECTORY
```

That proves your shell checks the current directory **before PATH**.

Then:

```
%testcmd
```

You **must get**:

```
PATH VERSION
```

That proves `%` skips the current directory and goes directly to `PATH`.

And finally:

/tmp/testcmd

should give:

PATH VERSION

That tests the literal-path rule too.

This is a very strong test because the outputs make it impossible to accidentally pass while using the wrong lookup order.